

Les exemples décrivent une bibliothèque : des auteurs (`Author`) et leurs livres (`Book`), chaque livre disponible en un nombre limité d'exemplaires (`copies`), des membres (`Member`) et leurs emprunts (`Loan`).

♦ Step 05 : relier deux modèles

`belongs_to` et `has_many`

```
bin/rails generate model Book author:references title:string copies:integer
```

Dans la migration, `author:references` devient `t.references :author, null: false, foreign_key: true` : une colonne `author_id`, un index, et une clé étrangère vers `authors`. Chaque côté de la relation se déclare dans son modèle :

```
class Book < ApplicationRecord
  belongs_to :author # le livre porte la colonne author_id
end

class Author < ApplicationRecord
  has_many :books, dependent: :destroy
end
```

Ces deux lignes génèrent les méthodes qui lisent et écrivent la relation :

```
zola = Author.create!(name: "Émile Zola")
zola.books # SELECT "books".* FROM "books" WHERE "books"."author_id" = 1
zola.books.count # SELECT COUNT(*) FROM "books" WHERE "books"."author_id" = 1
book.author # SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
zola.books.create!(title: "Germinal")
# INSERT INTO "books" ("author_id", "title", ...) VALUES (1, 'Germinal', ...)
```

`zola.books.create!` et `zola.books.build` remplissent `author_id` eux-mêmes.

`belongs_to` rend la relation obligatoire, sans que vous écriviez de validation :

```
book = Book.new(title: "Nana")
book.valid? # => false
book.errors.full_messages # => ["Author doit exister"]
```

Une relation facultative s'écrit `belongs_to :author, optional: true`.

`dependent: :destroy`

À la suppression d'un auteur, `dependent: :destroy` charge ses livres et les détruit un par un, avant de supprimer l'auteur :

```
Book Load      SELECT "books".* FROM "books" WHERE "books"."author_id" = 2
Book Destroy   DELETE FROM "books" WHERE "books"."id" = 3
Book Destroy   DELETE FROM "books" WHERE "books"."id" = 4
Author Destroy DELETE FROM "authors" WHERE "authors"."id" = 2
```

Sans `dependent`, la base refuse de supprimer un auteur qui a encore des livres : la clé étrangère lève `ActiveRecord::InvalidForeignKey`.

enum

Le statut d'un livre est stocké en entier, dans une colonne `t.integer :status, null: false, default: 0`. `enum` donne un nom à chaque valeur :

```
class Book < ApplicationRecord
  enum :status, { available: 0, on_loan: 1, withdrawn: 2 }
end

book.status          # => "available" : la base stocke 0
book.available?      # => true
book.on_loan!        # UPDATE "books" SET "status" = 1, ... WHERE "books"."id" = 1
Book.on_loan         # SELECT "books".* FROM "books" WHERE "books"."status" = 1
Book.not_on_loan     # ... WHERE "books"."status" != 1
Book.statuses        # => {"available" => 0, "on_loan" => 1, "withdrawn" => 2}
Book.new(status: :lost) # ArgumentError : 'lost' is not a valid status
```

`on_loan!` écrit en base tout de suite. Écrivez le hash avec ses entiers : chaque nom garde son entier quand vous ajoutez un statut.

Routes imbriquées

Un livre se crée depuis la page de son auteur, puis a sa propre adresse :

```
resources :authors do
  resources :books, only: %i[ new create ]
end
resources :books, except: %i[ new create ]
```

GET	/authors/:author_id/books/new	books#new	new_author_book_path(author)
POST	/authors/:author_id/books	books#create	author_books_path(author)
GET	/books/:id	books#show	book_path(book)

Dans le contrôleur, l'auteur vient de l'URL, jamais du formulaire :

```
class BooksController < ApplicationController
  before_action :set_author, only: %i[ new create ]

  def create
    @book = @author.books.build(book_params) # author_id est rempli par l'association
    # ...
  end

  private
  def set_author
    @author = Author.find(params.expect(:author_id))
  end
end
```

`form_with model: [@author, @book]` envoie le formulaire d'un nouveau livre à `POST /authors/1/books` ; pour un livre existant, `form_with model: @book` l'envoie à `PATCH /books/12`.

♦ Step 06 : many-to-many avec `has_many :through`

Un modèle de jointure

Un membre emprunte plusieurs livres, et un livre est emprunté par plusieurs membres. Entre les deux, une table `loans` : chaque

ligne relie un livre et un membre.

```
bin/rails generate model Member name:string email:string:uniq
bin/rails generate model Loan book:references member:references
```

```
class Loan < ApplicationRecord
  belongs_to :book
  belongs_to :member
end

class Book < ApplicationRecord
  has_many :loans, dependent: :destroy
  has_many :members, through: :loans
end

class Member < ApplicationRecord
  has_many :loans, dependent: :destroy
  has_many :books, through: :loans
end
```

`through` fait la jointure pour vous :

```
germinal.members
# SELECT "members".* FROM "members"
#   INNER JOIN "loans" ON "members"."id" = "loans"."member_id" WHERE "loans"."book_id" = 1
camille.books
# SELECT "books".* FROM "books"
#   INNER JOIN "loans" ON "books"."id" = "loans"."book_id" WHERE "loans"."member_id" = 1
```

Deux niveaux, et `distinct`

Les lecteurs d'un auteur passent par ses livres, puis par leurs emprunts :

```
class Author < ApplicationRecord
  has_many :books, dependent: :destroy
  has_many :loans, through: :books
  has_many :members, -> { distinct }, through: :loans
end

zola.members.count
# SELECT COUNT(DISTINCT "members"."id") FROM "members"
#   INNER JOIN "loans" ON "members"."id" = "loans"."member_id"
#   INNER JOIN "books" ON "loans"."book_id" = "books"."id" WHERE "books"."author_id" = 1
```

La jointure produit une ligne par emprunt : sans `distinct`, Camille, qui a emprunté deux livres de Zola, compterait deux fois.

La même règle, à deux endroits

```
class Member < ApplicationRecord
  validates :email, presence: true, uniqueness: true, format: { with: URI::MailTo::EMAIL_REGEXP }
end
```

`email:string:uniq` a aussi écrit dans la migration `add_index :members, :email, unique: true`.

- La **validation** cherche un doublon avant d'écrire (`SELECT 1 AS one FROM "members" WHERE "members"."email" = 'camille@example.test' LIMIT 1`) et donne un message lisible : « Email est déjà utilisé(e) ».
- L'**index unique** fait refuser le doublon par la base, même quand une écriture passe à côté des validations (`save(validate: false)`, `insert_all`, du SQL écrit à la main) : `ActiveRecord::RecordNotUnique`.

Sur deux colonnes, une validation avec `scope`, et un index sur le couple :

```
validates :title, uniqueness: { scope: :author_id, message: "existe déjà pour cet auteur" }

add_index :books, [ :author_id, :title ], unique: true      # dans une migration
```

L'option `message` remplace le message par défaut : « Title existe déjà pour cet auteur ».

find_or_initialize_by

```
member = Member.find_or_initialize_by(email: "denis@example.test")
# SELECT "members".* FROM "members" WHERE "members"."email" = 'denis@example.test' LIMIT 1
member.new_record?      # => true : pas trouvé, un nouvel objet, rien n'est écrit
member.name = "Denis Petit"
member.save              # INSERT INTO "members" ...
```

Contrairement à `find_or_create_by!` (step 02), rien n'est écrit : vous complétez l'objet, puis `save` le valide et l'enregistre. Dans un contrôleur d'emprunt :

```
@member = Member.find_or_initialize_by(email: member_params[:email])
@member.name = member_params[:name] if @member.new_record?
@loan = @book.loans.build(member: @member)

if @member.save && @loan.save
  redirect_to @book, notice: "Emprunt de #{@member.name} enregistré."
else
  # afficher @member.errors et @loan.errors
end
```

♦ Step 07 : méthode métier et validation personnalisée

Une méthode dans le modèle

```
class Book < ApplicationRecord
  def available_copies
    copies - loans.count      # SELECT COUNT(*) FROM "loans" WHERE "loans"."book_id" = 1
  end
end
```

Une méthode du modèle sert partout où l'objet est disponible : contrôleur, vue (`<%= @book.available_copies %>`), console, tests.

validate et errors.add

Aucune option de `validates` ne sait compter les emprunts. La règle s'écrit dans une méthode, déclarée avec `validate` (sans `s`):

```
class Loan < ApplicationRecord
  validate :book_has_copies, on: :create

  private
  def book_has_copies
    if book && book.available_copies <= 0
      errors.add(:book, "n'a plus d'exemplaire disponible")
    end
  end
end

loan = nana.loans.build(member: eve)
loan.valid?           # => false
loan.errors.full_messages # => ["Book n'a plus d'exemplaire disponible"]
loan.save             # => false, rien n'est écrit
```

- `errors.add` ajoute une erreur sur un attribut : dès qu'il y a une erreur, `valid?` et `save` renvoient `false`.
- `if book && :sans livre`, `book` vaut `nil`. `belongs_to` signale déjà « Book doit exister », le test évite une `NoMethodError`.
- `on: :create` : la règle ne s'applique qu'à un nouvel emprunt. Sans elle, un emprunt déjà enregistré deviendrait invalide dès que le livre n'a plus d'exemplaire disponible.

Un bouton, pas un lien

```
resources :loans, only: :destroy           # DELETE /loans/:id
```

```
<%= button_to "Rendre", loan, method: :delete,
  form: { data: { turbo_confirm: "Rendre ce livre ?" } } %>
```

produit un petit formulaire :

```
<form data-turbo-confirm="Rendre ce livre ?" class="button_to" method="post" action="/loans/1">
  <input type="hidden" name="_method" value="delete" />
  <button type="submit">Rendre</button>
  <input type="hidden" name="authenticity_token" value="..." />
</form>
```

- **Un formulaire, pas un lien** : un lien envoie un `GET`, que le navigateur peut précharger et qu'un robot peut suivre. Une action qui modifie des données passe par un formulaire.
- `_method` : un formulaire HTML n'envoie que `GET` ou `POST`. Rails lit le champ caché `_method` et traite la requête comme un `DELETE`.
- `authenticity_token` : un jeton que Rails ajoute à chaque formulaire et vérifie à la réception, pour refuser les requêtes envoyées depuis un autre site.
- `data-turbo-confirm` : Turbo, installé par défaut, demande confirmation avant d'envoyer le formulaire.

```
def destroy
  loan = Loan.find(params.expect(:id))
  loan.destroy!
  redirect_to loan.book, notice: "Livre rendu.", status: :see_other
end
```

`status: :see_other` (303) fait suivre la redirection en `GET`, quelle que soit la méthode de la requête d'origine. Le scaffold l'utilise déjà dans `update` et `destroy`.

♦ Step 08 : les callbacks

Un callback sur l'objet lui-même

Un callback est une méthode qu'Active Record appelle à un moment précis de la vie de l'objet :

```
class Member < ApplicationRecord
  before_validation :normalize_email

  private
    def normalize_email
      self.email = email.strip.downcase if email
    end
end
```

`self.email =` appelle le setter ; sans `self`, Ruby créerait une variable locale. Le callback s'exécute à chaque `valid?`, donc à chaque `save`, `create` ou `update`.

L'ordre des callbacks à la création :

```
before_validation → validations → after_validation
→ before_save → before_create → INSERT → after_create → after_save
→ COMMIT → after_commit
```

À la mise à jour, `before_update` et `after_update` remplacent les versions `create` ; à la suppression, `destroy` appelle `before_destroy`, le `DELETE`, puis `after_destroy`.

normalizes, aussi pour chercher

`before_validation` normalise l'email enregistré, pas celui qu'on cherche : `find_or_initialize_by(email: "Camille@Example.test")` ne trouve pas `camille@example.test`, prépare un nouveau membre, que la validation d'unicité refuse. `normalizes` s'applique à l'affectation **et** aux recherches :

```
class Member < ApplicationRecord
  normalizes :email, with: ->(email) { email.strip.downcase }
end

Member.new(email: " Eve@Example.TEST ").email      # => "eve@example.test", dès l'affectation
Member.where(email: " CAMILLE@Example.test").to_sql
# SELECT "members".* FROM "members" WHERE "members"."email" = 'camille@example.test'
```

Un callback qui modifie un autre objet

Un livre passe à « prêté » quand son dernier exemplaire part, et redevient disponible quand un exemplaire revient :

```
class Book < ApplicationRecord
  def refresh_status
    if available? && available_copies.zero?
      on_loan!
    elsif on_loan? && available_copies.positive?
      available!
    end
  end
end

class Loan < ApplicationRecord
  after_create :refresh_book_status
  after_destroy :refresh_book_status

  private
  def refresh_book_status
    book.refresh_status
  end
end
```

Le log d'un `loan.destroy` montre tout ce que le callback déclenche :

```
Loan Destroy  DELETE FROM "loans" WHERE "loans"."id" = 2
Book Load    SELECT "books".* FROM "books" WHERE "books"."id" = 2 LIMIT 1
Loan Count    SELECT COUNT(*) FROM "loans" WHERE "loans"."book_id" = 2
Book Update   UPDATE "books" SET "status" = 0, "updated_at" = ... WHERE "books"."id" = 2
```

Un callback s'exécute à chaque création ou suppression, d'où qu'elle vienne : contrôleur, console, seeds, tests. Mais `delete`, `update_column` et `update_all` écrivent sans validations ni callbacks : après un `loan.delete`, le statut du livre n'est pas recalculé.

Ce que le callback cache

```
def destroy # LoansController
  loan = Loan.find(params.expect(:id))
  loan.destroy!
  redirect_to loan.book, notice: "Livre rendu.", status: :see_other
end
```

Rien dans cette action ne dit que le livre peut changer de statut : l'effet est déclaré dans `Loan`, loin de l'appel qui le déclenche.

- `normalize_email` ne touche que l'objet en cours d'enregistrement : c'est l'usage le plus sûr.
- `refresh_book_status` modifie **un autre objet** : pratique, mais invisible pour qui lit le contrôleur.

Dans un modèle que vous n'avez pas écrit, cherchez les callbacks en premier : les lignes `before_...` et `after_...`, et les **dependent** : des associations, qui en posent aussi.

♦ Step 09 : scopes et filtres

Un scope

```
class Book < ApplicationRecord
  scope :long, -> { where(pages: 500..) }
end

Book.long # SELECT "books".* FROM "books" WHERE "books"."pages" >= 500
```

Un scope donne un nom à une requête. C'est une méthode de classe qui renvoie une relation, que l'on peut compléter. Les intervalles sans borne donnent les comparaisons : `500..` pour `>= 500`, `...Time.current` pour `< Time.current`.

Enchaîner, sans requête

Les scopes s'enchaînent, avec ceux que génère un `enum` :

```
Book.long.available.to_sql
# SELECT "books".* FROM "books" WHERE "books"."pages" >= 500 AND "books"."status" = 0
```

Une relation est lazy : chaque scope ajoute une condition, et la requête ne part qu'au moment où l'on a besoin du résultat.

```
books = Book.long # aucune requête
books.class # => Book::ActiveRecord_Relation
books.count # SELECT COUNT(*) FROM "books" WHERE "books"."pages" >= 500
books.each { |book| ... } # SELECT "books".* FROM "books" WHERE "books"."pages" >= 500
```

Un scope avec un argument

```
scope :with_status, ->(status) { where(status: status) if status.present? }
scope :by_author, ->(author_id) { where(author_id: author_id) if author_id.present? }

Book.with_status("on_loan").to_sql # SELECT "books".* FROM "books" WHERE "books"."status" = 1
Book.with_status(nil).to_sql # SELECT "books".* FROM "books"
```

Quand la condition est fausse, le bloc renvoie `nil`, et le scope renvoie alors toutes les lignes : la chaîne continue sans vérifier chaque filtre.

Filtrer une liste

Un formulaire envoyé en `GET` place ses champs dans l'URL :

```
<%= form_with url: books_path, method: :get do |form| %>
  <%= form.select :status, Book.statuses.keys,
    { include_blank: "Tous les statuts", selected: params[:status] } %>
  <%= form.collection_select :author_id, Author.order(:name), :id, :name,
    { include_blank: "Tous les auteurs", selected: params[:author_id] } %>
  <%= form.submit "Filtrer", name: nil %>
<% end %>
```

Il envoie par exemple `GET /books?status=available&author_id=1`. Le contrôleur enchaîne les scopes :

```
def index
  @books = Book.order(:title).with_status(params[:status]).by_author(params[:author_id])
end
```

Les filtres ne sont pas enregistrés en base : pas besoin de `params.expect`, chaque scope reçoit une seule valeur.

Le N+1 et includes

La liste affiche `book.author.name` pour chaque livre. Le log de la requête :

```
Book Load      SELECT "books".* FROM "books" ORDER BY "books"."title" ASC
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 3 LIMIT 1
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" = 2 LIMIT 1
CACHE Author Load SELECT "authors".* FROM "authors" WHERE "authors"."id" = 1 LIMIT 1
CACHE Author Load SELECT "authors".* FROM "authors" WHERE "authors"."id" = 2 LIMIT 1
Completed 200 OK in 32ms (Views: 19.6ms | ActiveRecord: 1.7ms (6 queries, 2 cached) | GC: 0.0ms)
```

Une requête pour la liste, puis une par ligne : c'est le **N+1**. **CACHE** signale une requête identique à une précédente, resservie sans interroger la base, le temps de la requête HTTP. Invisible avec cinq livres, coûteux avec cinq mille. **includes** charge les auteurs en même temps que les livres :

```
@books = Book.includes(:author).order(:title)
```

```
Book Load      SELECT "books".* FROM "books" ORDER BY "books"."title" ASC
Author Load    SELECT "authors".* FROM "authors" WHERE "authors"."id" IN (1, 3, 2)
Completed 200 OK in 30ms (Views: 17.7ms | ActiveRecord: 1.5ms (2 queries, 0 cached) | GC: 0.0ms)
```

Deux requêtes, quel que soit le nombre de livres. Dans le log, sous chaque requête SQL, une ligne qui commence par `↳` désigne le code qui l'a lancée, par exemple `app/views/books/index.html.erb:5`.

✦ Pour aller plus loin : la documentation officielle

Step 05 - Les sessions d'un atelier

- [Active Record Migrations - Column Modifiers](https://guides.rubyonrails.org/active_record_migrations.html#column-modifiers)
guides.rubyonrails.org/active_record_migrations.html#column-modifiers
- [Active Record Associations](https://guides.rubyonrails.org/association_basics.html)
guides.rubyonrails.org/association_basics.html
- [ActiveRecord::Enum](https://api.rubyonrails.org/classes/ActiveRecord/Enum.html)
api.rubyonrails.org/classes/ActiveRecord/Enum.html
- [Nested Resources](https://guides.rubyonrails.org/routing.html#nested-resources)
guides.rubyonrails.org/routing.html#nested-resources
- [Action Controller Overview](https://guides.rubyonrails.org/action_controller_overview.html)
guides.rubyonrails.org/action_controller_overview.html

Step 06 - Participants et inscriptions

- [The has_many :through Association](https://guides.rubyonrails.org/association_basics.html#has-many-through)
guides.rubyonrails.org/association_basics.html#has-many-through
- [Uniqueness](https://guides.rubyonrails.org/active_record_validations.html#uniqueness)
guides.rubyonrails.org/active_record_validations.html#uniqueness
- [find_or_initialize_by \(API\)](https://api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_initialize_by)
api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_initialize_by

Step 07 - La jauge

- [Custom Methods](https://guides.rubyonrails.org/active_record_validations.html#custom-methods)
guides.rubyonrails.org/active_record_validations.html#custom-methods
- [errors.add](https://api.rubyonrails.org/classes/ActiveModel/Errors.html#method-i-add)
api.rubyonrails.org/classes/ActiveModel/Errors.html#method-i-add
- [button_to](https://api.rubyonrails.org/classes/ActionView/Helpers/UrlHelper.html#method-i-button_to)
api.rubyonrails.org/classes/ActionView/Helpers/UrlHelper.html#method-i-button_to
- [Turbo - Requiring Confirmation](https://turbo.hotwired.dev/handbook/drive#requiring-confirmation-for-a-visit)
turbo.hotwired.dev/handbook/drive#requiring-confirmation-for-a-visit

Step 08 - Les callbacks

- [Active Record Callbacks](#)

guides.rubyonrails.org/active_record_callbacks.html

- [normalizes \(API\)](https://api.rubyonrails.org/classes/ActiveModel/Attributes/Normalization/ClassMethods.html#method-i-normalizes)
api.rubyonrails.org/classes/ActiveModel/Attributes/Normalization/ClassMethods.html#method-i-normalizes
- [Available Callbacks](https://guides.rubyonrails.org/active_record_callbacks.html#available-callbacks)
guides.rubyonrails.org/active_record_callbacks.html#available-callbacks

Step 09 - Scopes et filtres

- [Scopes](https://guides.rubyonrails.org/active_record_querying.html#scopes)
guides.rubyonrails.org/active_record_querying.html#scopes
- [Form Helpers - A Generic Search Form](https://guides.rubyonrails.org/form_helpers.html#a-generic-search-form)
guides.rubyonrails.org/form_helpers.html#a-generic-search-form
- [N+1 Queries Problem](https://guides.rubyonrails.org/active_record_querying.html#n-1-queries-problem)
guides.rubyonrails.org/active_record_querying.html#n-1-queries-problem
- [Eager Loading Associations](https://guides.rubyonrails.org/active_record_querying.html#eager-loading-associations)
guides.rubyonrails.org/active_record_querying.html#eager-loading-associations